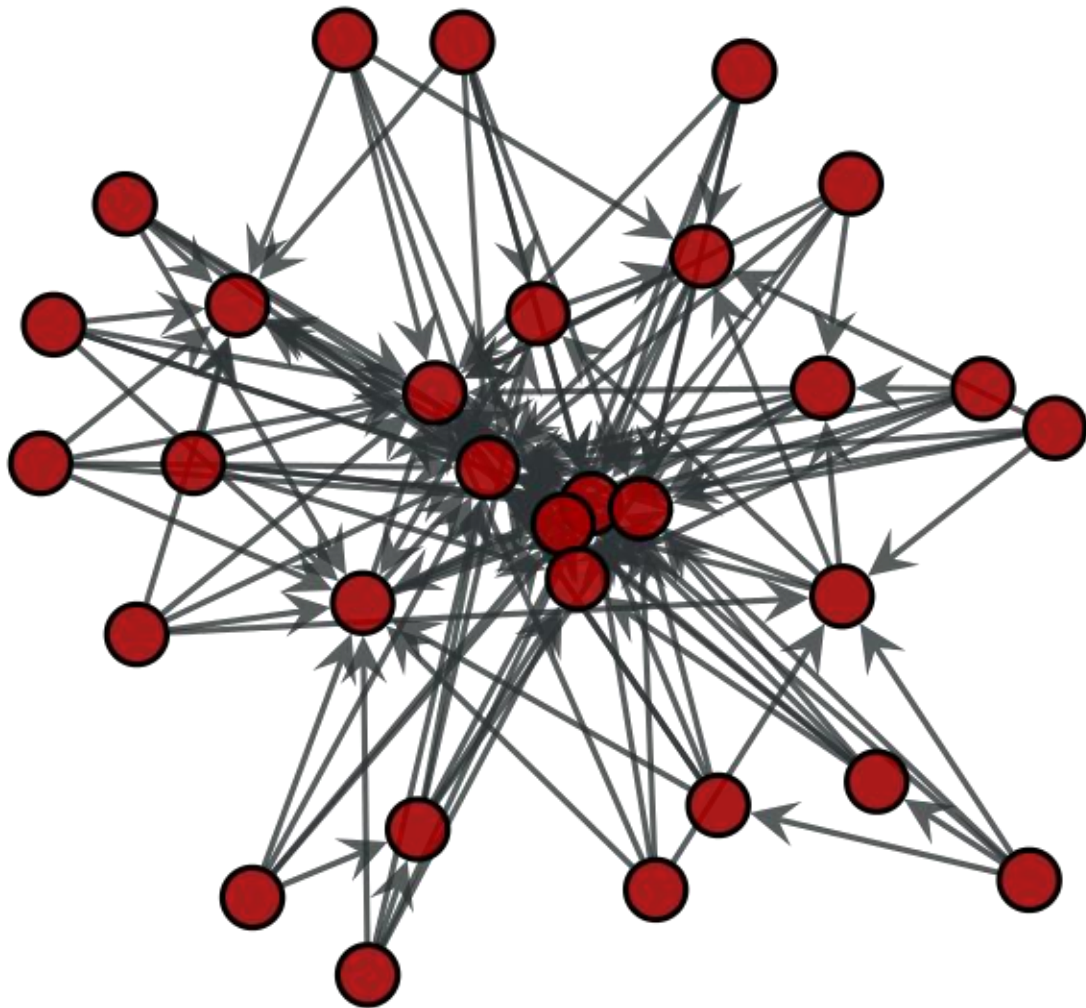




ClinicalAI: Instruction Manual

HOW TO BUILD THE CLINICALAI KNOWLEDGE GRAPH, AND QUERY IT WITH NATURAL LANGUAGE USING A LARGE LANGUAGE MODEL

Prerana S K | ClinicalAI | 19-07-2024

Brief Overview of the Project

90% of clinical experiments fail, due to insufficient background work and research. This can be improved with the help of better pre-planning and more informed decision making. ClinicalAI is a project that aims to bring together the logical deduction capabilities and structure provided by a KG (Knowledge Graph) with the natural language interpretation capabilities of an LLM. This is a step-by-step instruction manual that goes through all the steps (from scratch) of completing the set-up for this project. In the following sections, we first see how to collect and clean data from <https://www.clinicaltrials.gov/>. Then we see how a schema is built on Neo4j's AuraDB, and finally how to integrate it with GPT-4.

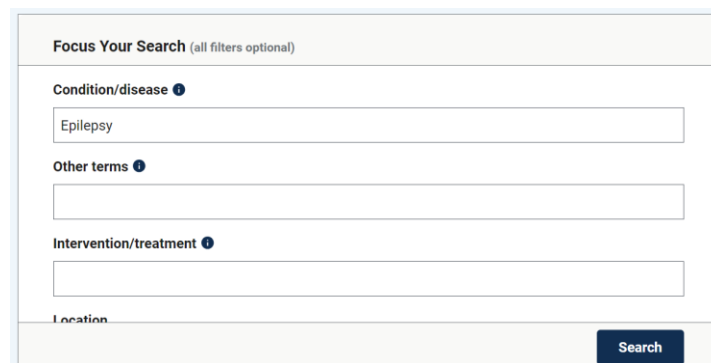
*** All code can be found at <https://github.com/Narpear/HeCiX>

SECTION-A (COLLECTING AND CLEANING THE DATASET)

A) Collecting the Dataset

The first step of our project involves collecting our data and cleaning it. For this proof-of-concept project, we acquire data from <https://www.clinicaltrials.gov/> of six diseases, 200 records each. The six diseases we use are vitiligo, melanoma, atopic dermatitis, alopecia areata, vitiligo, and epilepsy. (On reverse engineering, we realized that these have common nodes such as principal investigator, conditions, etc. and therefore picked these six diseases.)

To do this, we visit <https://www.clinicaltrials.gov/> and type in the name of the disease of interest. In our case, this is epilepsy. The other fields can be left blank.



Focus Your Search (all filters optional)

Condition/disease ①
Epilepsy

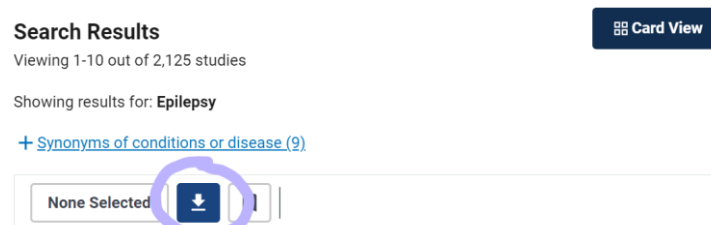
Other terms ①

Intervention/treatment ①

Location

Search

We then click on the download button on the page.



Search Results

Viewing 1-10 out of 2,125 studies

Showing results for: Epilepsy

+ [Synonyms of conditions or disease \(9\)](#)

None Selected [Download Icon]

File Format



Results to Download

 Top 200

[Select all](#) / [De-select all](#)

The downloaded .csv file should look something like this

[illegible]

PAGE 2

B) Scraping Principal-Investigators from Individual Webpages

To scrape this data from webpages of each clinical trial, we write some basic python code. This code can be found at https://github.com/Narpear/HeCiX/tree/main/clinical_trials.

There are four python files here that help with scraping data we need.

- **download_html.py**
(https://github.com/Narpear/HeCiX/blob/main/clinical_trials/download_html.py)
This function downloads and saves the formatted HTML content of a specific clinical trial page from clinicaltrials.gov using its NCT number.
- **get_div.py**
(https://github.com/Narpear/HeCiX/blob/main/clinical_trials/get_div.py)
This function extracts and returns the principal investigator names from HTML pages locally saved by download_html.py, identified by its NCT number.
- **delete_html.py**
(https://github.com/Narpear/HeCiX/blob/main/clinical_trials/delete_html.py)
This function deletes a specific HTML file named after an NCT number if it exists in the current directory, or prints a message if the file does not exist.
- **append_pis.py**
(https://github.com/Narpear/HeCiX/blob/main/clinical_trials/append_pis.py)
This script processes a CSV file of clinical trials, downloads each trial's webpage, extracts principal investigator names, updates a dataframe with this information, deletes the downloaded HTML, and saves the results to a new CSV file.

The file titled append_pis.py is the main file that is to be run, and the others are basically helper functions. The paths used within the code should be changed, to match the locations of the files on other systems.

C) Combining Collected Data and Cleaning Up

The data collected from different diseases are to be put in a single .csv file. This is done by appending the name of the disease the trial is associated with, at the beginning of each record. This can be found here: https://github.com/Narpear/HeCiX/blob/main/clinical_trials/6_Diseases_With_PI%2o-%2oSheet1.csv.

The columns with multiple values (principal investigators, conditions, locations, and interventions) are split into separate columns. These cells are left blank for records that do not have multiple values. This can be found here: <https://colab.research.google.com/drive/12cW196ALr-UwzGjPGPIFM4trC1o6gLkK?usp=sharing> and the code to separate columns can be found here <https://colab.research.google.com/drive/12cW196ALr-UwzGjPGPIFM4trC1o6gLkK?usp=sharing>.

Finally, special and unidentified characters are replaced with true values from the actual webpages, by manual cleaning.

The processed .csv file that's ready for use can be found at https://github.com/Narpear/HeCiX/blob/main/clinical_trials/Cleaned%2oFinal%2oDataset.csv

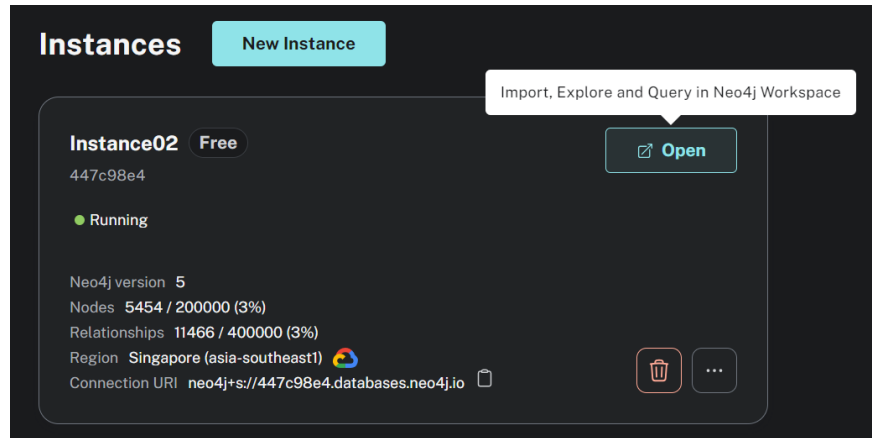
This completes dataset collection and cleaning.

SECTION B (BUILDING THE KNOWLEDGE GRAPH)

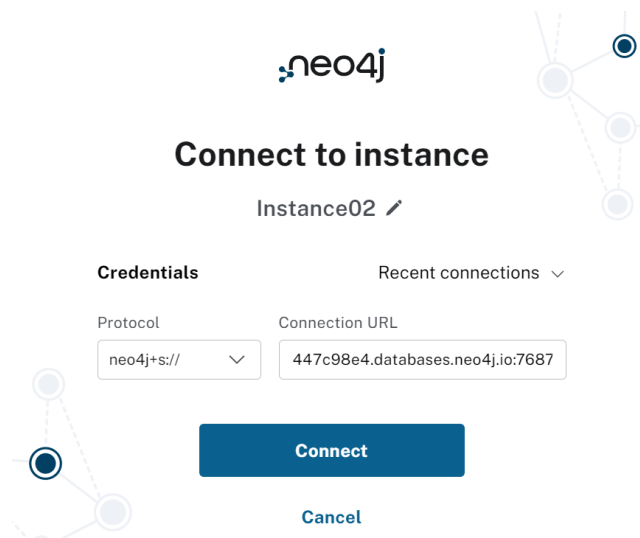
The following steps are all done on AuraDB (<https://neo4j.com/cloud/platform/aura-graph-database/>). First, you must create an instance and note down the username (which is usually neo4j) and the password, or change the password if needed and store it securely.

A) Constructing the Schema

First, we open our instance of AuraDB.

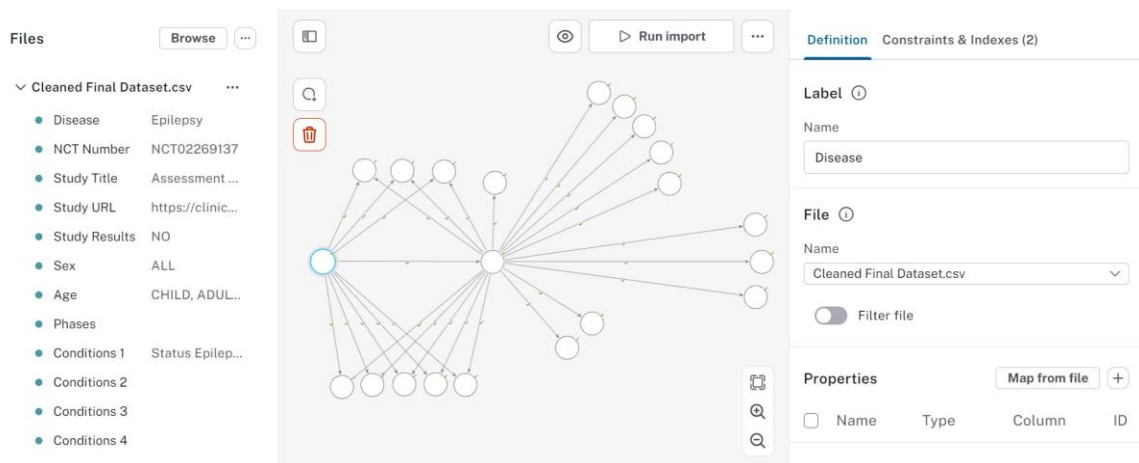


The URI of the instance is the protocol, followed by the connection URL. Make sure to store the URI safely as well, since it is important for connecting to the database later.



For example, the URI here would be neo4j+s://447c98e4.databases.neo4j.io:7687

Import the .csv file and build the schema and map the columns of the csv to the nodes and edges of the knowledge graph. This is done by creating a node/relationship, then double clicking on it and adding the required properties to it.



Once the schema is built, 'Run import' to populate the schema and finish constructing the knowledge graph.

This completes building the knowledge graph.

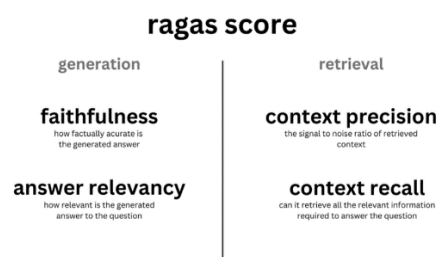
SECTION C (CONNECTING THE KNOWLEDGE GRAPH TO THE LLM)

https://github.com/Narpear/HeCiX/blob/main/KG_Aug_LLM_GPT4.ipynb is the code file used to connect the LLM to the Knowledge Graph. The KG once connected to GPT-4, can be queried using natural language. LangChain is used to connect the KG with the LLM. Natural language queries sent to the system are converted into CQL (Cypher Query Language) by GPT-4. GraphCypherQACChain is a tool from the LangChain ecosystem that's used to query graph databases. This sends the cypher query to the AuraDB database and retrieves the result. The answer is stored as the context. The cell output is captured, and send to GPT-4 once again (along with the question), for further formatting and receiving the output in a more polished and human-readable (natural language) format.

Before running the entire code, make sure to save environment variables (and also enter the OpenAI API key).

SECTION D (EVALUATING THE RAG MODEL)

https://github.com/Narpear/HeCiX/blob/main/Prer_RAGAS_Evaluation.ipynb contains code to evaluate the RAG model using the RAGAS library. Here are the metrics we have used, and what they mean:



And we are done!